

Java Collections: Queue, Concurrent, and Utilities - Cheat Sheet

Queue interface

The `Queue` interface models FIFO (first-in, first-out) by default. The catch is every operation has 2 flavors: one that throws on failure and one that returns a special value.

Operation	Throws on failure	Returns special value
Insert	<code>add(e)</code>	<code>offer(e)</code>
Remove	<code>remove()</code>	<code>poll()</code> (returns null)
Examine	<code>element()</code>	<code>peek()</code> (returns null)

The throwing variants come from `Collection`. The `offer`, `poll`, `peek` group is what you usually want, especially with bounded queues where insert can legitimately fail.

```
Queue<String> queue = new LinkedList<>();
queue.offer("task-1");
queue.offer("task-2");

String head = queue.peek();           // "task-1", queue unchanged
String taken = queue.poll();          // "task-1", removed
String empty = queue.poll();          // returns null if empty, no exception
```

Use throwing variants only when “empty” is a real error in your code path (rare). For producer-consumer loops, polling and checking null is cleaner.

Deque interface

`Deque` (double-ended queue) extends `Queue` and adds operations on both ends.

```
Deque<Integer> deque = new ArrayDeque<>();
deque.addFirst(1);
deque.addLast(2);
deque.addLast(3);
// front -> [1, 2, 3] <- back

deque.peekFirst();    // 1
deque.peekLast();     // 3
deque.pollFirst();    // 1, removed
deque.pollLast();     // 3, removed
```

`Deque` also gives you stack operations. `push(e)` is `addFirst(e)`, `pop()` is `pollFirst()`. This replaces both the legacy `Stack` class and the awkward use of `LinkedList` as a queue.

```
Deque<String> stack = new ArrayDeque<>();
stack.push("frame-1");
stack.push("frame-2");
stack.pop(); // "frame-2"
```

If you see `new Stack<>()` in code review, flag it. `ArrayDeque` is faster and doesn't inherit `Vector`'s synchronized overhead.

ArrayDeque

`ArrayDeque` is a resizable circular array. It's the default choice for a non-concurrent queue or stack in modern Java. It runs $O(1)$ amortized for `addFirst`, `addLast`, `pollFirst`, `pollLast`. It's faster than `LinkedList` for queue and deque ops because of better cache locality and no per-node allocation, and it's faster than the legacy `Stack` class for LIFO usage. It doesn't allow nulls since the implementation uses null as an empty-slot sentinel.

```
Deque<Request> pending = new ArrayDeque<>();
pending.offer(req1);
pending.offer(req2);

while (!pending.isEmpty()) {
    Request r = pending.poll();
    process(r);
}
```

If you need null elements, you're picking the wrong structure anyway. Use `Optional` or a sentinel object.

PriorityQueue

`PriorityQueue` is a binary min-heap backed by an array. It orders elements by natural order or a supplied `Comparator`. It doesn't do FIFO, and order among equal-priority elements is unspecified. `peek()` returns the smallest element in $O(1)$. `offer(e)` and `poll()` are $O(\log n)$. `remove(Object)` and `contains(Object)` are $O(n)$ since they do a linear scan.

```

class Task {
    String name;
    Instant dueAt;
    Task(String name, Instant dueAt) { this.name = name; this.dueAt = dueAt; }
}

PriorityQueue<Task> queue = new PriorityQueue<>(
    Comparator.comparing(t -> t.dueAt)
);

queue.offer(new Task("send-email", Instant.now().plusSeconds(60)));
queue.offer(new Task("retry-payment", Instant.now().plusSeconds(10)));
queue.offer(new Task("cleanup", Instant.now().plusSeconds(300)));

while (!queue.isEmpty()) {
    Task next = queue.poll();           // earliest due first
    run(next);
}

```

Common uses: top-N problems (keep a bounded min-heap and evict the smallest), Dijkstra-style algorithms, in-memory task scheduling.

Don't iterate a `PriorityQueue` and expect sorted order. The iterator visits elements in heap order, which isn't sorted. To get sorted output, poll repeatedly.

BlockingQueue family

`BlockingQueue` (in `java.util.concurrent`) is the producer/consumer toolkit. It adds blocking variants on top of the `Queue` interface.

Operation	Throws	Returns special	Blocks	Times out
Insert	<code>add(e)</code>	<code>offer(e)</code>	<code>put(e)</code>	<code>offer(e, t, unit)</code>
Remove	<code>remove()</code>	<code>poll()</code>	<code>take()</code>	<code>poll(t, unit)</code>
Examine	<code>element()</code>	<code>peek()</code>	n/a	n/a

`put` blocks if the queue is full. `take` blocks if it's empty. That's the whole point.

Implementations

- `ArrayBlockingQueue` is bounded, backed by a circular array, single lock with optional fairness.
- `LinkedBlockingQueue` is optionally bounded (default `Integer.MAX_VALUE`), uses separate locks for head and tail (higher throughput under contention).
- `PriorityBlockingQueue` is unbounded, heap-based, blocking on `take` only (since it's unbounded, `put` never blocks).

- `DelayQueue` elements implement `Delayed`. `take()` blocks until the head's delay expires. Good for scheduled work without spinning up a `ScheduledExecutor`.
- `SynchronousQueue` has zero capacity. Every `put` blocks until a `take` pairs with it (and vice versa). Used internally by `Executors.newCachedThreadPool()`.
- `LinkedTransferQueue` extends `BlockingQueue` with `transfer(e)`, which blocks until a consumer takes the element.

Classic producer-consumer

```
BlockingQueue<Order> orders = new ArrayBlockingQueue<>(100);

// Producer
Thread producer = new Thread(() -> {
    try {
        while (true) {
            Order o = receiveFromHttp();
            orders.put(o); // blocks if full
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});

// Consumer
Thread consumer = new Thread(() -> {
    try {
        while (true) {
            Order o = orders.take(); // blocks if empty
            process(o);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});
```

For bounded backpressure (HTTP intake feeding a slow worker), `ArrayBlockingQueue` is the default. For high-throughput, multi-producer/multi-consumer scenarios, `LinkedBlockingQueue` often wins because of its 2-lock design.

TransferQueue

`TransferQueue` extends `BlockingQueue` with a “hand off directly” semantic. `put(e)` enqueues and returns, so the consumer takes whenever. `transfer(e)` blocks until a consumer actually takes the element.

`LinkedTransferQueue` is the only standard implementation. Use it when you want to know the consumer has received the work, instead of just knowing you handed it to a buffer.

```
LinkedTransferQueue<Job> queue = new LinkedTransferQueue<>();

// blocks until some worker takes this job
queue.transfer(new Job("urgent"));
```

Concurrent map implementations

ConcurrentHashMap

The default thread-safe map for almost everything. It uses bucket-level locking (Java 8+ uses CAS plus synchronized blocks per bin, not stripe locks like Java 7). Iterators are weakly consistent, so they don't throw `ConcurrentModificationException` and may or may not reflect concurrent modifications. `compute`, `computeIfAbsent`, `computeIfPresent`, and `merge` are atomic on the affected key. It doesn't allow null keys or null values, since you can't tell "missing" from "present and null" with `get`.

Counting words from multiple threads:

```
ConcurrentHashMap<String, Long> counts = new ConcurrentHashMap<>();

ExecutorService pool = Executors.newFixedThreadPool(8);
for (String doc : documents) {
    pool.submit(() -> {
        for (String word : tokenize(doc)) {
            counts.merge(word, 1L, Long::sum); // atomic increment per key
        }
    });
}
pool.shutdown();
pool.awaitTermination(1, TimeUnit.MINUTES);
```

`merge` here is the right call. With a plain `HashMap` and external `synchronized`, you'd serialize every increment. With `ConcurrentHashMap.merge`, only writers on the same bucket contend.

`computeIfAbsent` is the canonical way to lazily build per-key state:

```
ConcurrentHashMap<String, List<Event>> buckets = new ConcurrentHashMap<>();
buckets.computeIfAbsent(userId, k -> new CopyOnWriteArrayList<>()).add(event);
```

That whole expression is atomic with respect to the key. With `synchronizedMap`, you'd need to lock the map yourself.

ConcurrentSkipListMap

A sorted, thread-safe `NavigableMap`. Backed by a skip list (probabilistic balanced structure). Use it when you need both ordering and concurrency. `get`, `put`, `remove`, `containsKey` are $O(\log n)$ on

average. All `NavigableMap` ops work concurrently: `firstKey`, `lastKey`, `floorKey`, `ceilingKey`, `subMap`, `headMap`, `tailMap`. Iterators are weakly consistent.

```
ConcurrentSkipListMap<Long, Event> byTimestamp = new ConcurrentSkipListMap<>();
byTimestamp.put(eventTs, event);

// recent events from the last 10 seconds, no locking needed
Map<Long, Event> recent = byTimestamp.tailMap(System.currentTimeMillis() - 10_000);
```

`TreeMap` wrapped in `Collections.synchronizedSortedMap` is the alternative, but `ConcurrentSkipListMap` scales better under contention.

ConcurrentSkipListSet

The set companion. Same trade-offs, same use case.

Concurrent lock-free collections

When you want non-blocking throughput over the blocking producer/consumer pattern.

`ConcurrentLinkedQueue` is unbounded, lock-free FIFO based on the Michael & Scott algorithm.

`ConcurrentLinkedDeque` does the same thing, double-ended.

These don't have `put` or `take`. They're CAS-based, so writers never block each other (they retry). Use when consumers shouldn't park waiting for data (busy event loops, low-latency pipelines).

```
ConcurrentLinkedQueue<Event> queue = new ConcurrentLinkedQueue<>();

// producer
queue.offer(event);

// consumer (in a loop, doesn't block)
Event e = queue.poll();
if (e != null) {
    handle(e);
}
```

Caveat: `size()` on `ConcurrentLinkedQueue` is $O(n)$ and not atomic with concurrent updates. Don't poll size in a hot loop.

Copy-on-write collections

`CopyOnWriteArrayList` makes every mutation copy the entire backing array. Reads are lock-free against a stable snapshot. Reads do zero locking with snapshot semantics. Writes copy the whole array, which is very expensive. Iterators are snapshot-based, so no

`ConcurrentModificationException`, but they won't see writes that happened after iteration started.

Best fit: read-mostly listener lists, observers, event subscribers. Rare writes (configuration reload, registering a listener), frequent reads (firing events).

```
CopyOnWriteArrayList<Listener> listeners = new CopyOnWriteArrayList<>();

// hot path: fire events to all listeners
for (Listener l : listeners) {
    l.onEvent(event);
}

// rare path: add a listener (this copies the array)
listeners.add(newListener);
```

If you're writing as often as reading, this is the wrong structure. Use `ConcurrentLinkedQueue` or a real concurrent collection.

`CopyOnWriteArraySet` is the set version, built on top of `CopyOnWriteArrayList`.

Synchronized wrappers (legacy)

`Collections.synchronizedList`, `synchronizedMap`, `synchronizedSet` wrap an existing collection and lock the whole thing on every operation.

```
List<String> list = Collections.synchronizedList(new ArrayList<>());
list.add("a"); // locks the wrapper

Map<String, Integer> map = Collections.synchronizedMap(new HashMap<>());
```

2 big issues:

1. Compound operations aren't atomic. `if (!map.containsKey(k)) map.put(k, v);` is a race even with `synchronizedMap`. You need an external `synchronized` block.
2. Iteration requires manual synchronization:

```
synchronized (list) {
    for (String s : list) {
        // safe only inside this block
    }
}
```

These wrappers exist for backward compatibility. For new code, use `ConcurrentHashMap`, `CopyOnWriteArrayList`, or `ConcurrentSkipListMap`. They give you finer-grained locking and atomic compound ops.

Why concurrent collections beat synchronized wrappers

3 concrete reasons:

1. Finer-grained locking. `ConcurrentHashMap` locks per bucket. `synchronizedMap` locks the whole map. Under contention, the difference is large.
2. Lock-free reads. `ConcurrentHashMap.get` doesn't take a lock in the common case. `ConcurrentLinkedQueue` is fully lock-free.
3. Atomic compound operations. `computeIfAbsent`, `merge`, `putIfAbsent` are atomic on the key. Doing the same thing with `synchronizedMap` requires you to lock the map yourself, and people get it wrong constantly.

Collections utility class

`java.util.Collections` is a grab bag of static helpers. The ones you'll actually use:

Sorting and searching

```
Collections.sort(list); // natural order
Collections.sort(list, Comparator.reverseOrder()); // custom
int idx = Collections.binarySearch(list, key); // list must be sorted
```

Reordering

```
Collections.reverse(list);
Collections.shuffle(list);
Collections.rotate(list, 2); // shift right by 2
Collections.swap(list, 0, 3);
```

Min, max, frequency

```
String smallest = Collections.min(list);
String biggest = Collections.max(list, Comparator.comparingInt(String::length));
int howMany = Collections.frequency(list, "target");
```

Wrappers

```
List<String> readOnly = Collections.unmodifiableList(list);
Map<String, Integer> safe = Collections.synchronizedMap(map);
List<String> typed = Collections.checkedList(list, String.class); // runtime
                        type check
```


Empty and singleton

```
List<String> none = Collections.emptyList();
Set<String> none2 = Collections.emptySet();
Map<String, Integer> none3 = Collections.emptyMap();

List<String> one = Collections.singletonList("only");
Set<String> one2 = Collections.singleton("only");
Map<String, Integer> one3 = Collections.singletonMap("key", 1);
```

Fill, copy, replace, nCopies

```
Collections.fill(list, "default");
List<Integer> tenZeros = new ArrayList<>(Collections.nCopies(10, 0));
Collections.replaceAll(list, "old", "new");
```

`checkedList`, `checkedSet`, `checkedMap` are underrated. They wrap a collection so any insert of the wrong type throws immediately, even through raw-type APIs. Useful when a generic collection is passed across module boundaries.

Arrays utility class

`java.util.Arrays` is the array companion.

```

// Fixed-size list backed by the array
List<String> view = Arrays.asList("a", "b", "c");
// view.add("d") throws UnsupportedOperationException
// view.set(0, "z") works and updates the backing array

// Sort
Arrays.sort(arr);
Arrays.parallelSort(arr);    // forks to common ForkJoinPool for big arrays

// Binary search (array must be sorted)
int idx = Arrays.binarySearch(arr, key);

// Stream
IntStream s = Arrays.stream(intArr);
Stream<String> s2 = Arrays.stream(strArr);

// Copy
String[] head = Arrays.copyOf(arr, 5);
String[] mid = Arrays.copyOfRange(arr, 2, 6);

// Equality
Arrays.equals(a, b);          // shallow, element-by-element
Arrays.deepEquals(a, b);      // recurses into nested arrays

Arrays.hashCode(arr);         // shallow
Arrays.deepHashCode(arr);     // nested

```

The `Arrays.asList` trap shows up in interviews. It returns a fixed-size `List` backed by the array. Calling `add` or `remove` throws `UnsupportedOperationException`. Calling `set` updates the backing array. If you want a real `ArrayList`, wrap it: `new ArrayList<>(Arrays.asList(...))`.

Stream collectors

`java.util.stream.Collectors` is the bridge from streams back to collections. The methods you'll use over and over:

To collection

```

List<User> active = users.stream()
    .filter(User::isActive)
    .collect(Collectors.toList());

List<User> readOnly = users.stream()
    .collect(Collectors.toUnmodifiableList());

Set<String> emails = users.stream()
    .map(User::getEmail)
    .collect(Collectors.toSet());

```

To map (with duplicate-key handling)

```
Map<String, User> byEmail = users.stream()
    .collect(Collectors.toMap(User::getEmail, u -> u));

// duplicate keys throw IllegalStateException unless you supply a merge function
Map<String, User> byCity = users.stream()
    .collect(Collectors.toMap(
        User::getCity,
        u -> u,
        (existing, replacement) -> existing    // keep the first one
    ));
```

Grouping

```
// Group users by status: Map<Status, List<User>>
Map<Status, List<User>> byStatus = users.stream()
    .collect(Collectors.groupingBy(User::getStatus));

// Group + count: Map<Status, Long>
Map<Status, Long> countByStatus = users.stream()
    .collect(Collectors.groupingBy(User::getStatus, Collectors.counting()));

// Group + map to email: Map<Status, List<String>>
Map<Status, List<String>> emailsByStatus = users.stream()
    .collect(Collectors.groupingBy(
        User::getStatus,
        Collectors.mapping(User::getEmail, Collectors.toList())
    ));
```

Partitioning (boolean grouping)

```
Map<Boolean, List<Order>> shippedOrNot = orders.stream()
    .collect(Collectors.partitioningBy(Order::isShipped));

List<Order> shipped = shippedOrNot.get(true);
List<Order> pending = shippedOrNot.get(false);
```

Numeric aggregates

```
long total = orders.stream().collect(Collectors.counting());
int sum = orders.stream().collect(Collectors.summingInt(Order::getCents));
double avg = orders.stream().collect(Collectors.averagingInt(Order::getCents));

IntSummaryStatistics stats = orders.stream()
    .collect(Collectors.summarizingInt(Order::getCents));
// stats.getMin(), getMax(), getAverage(), getSum(), getCount()
```

Joining

```
String csv = users.stream()
    .map(User::getEmail)
    .collect(Collectors.joining(", "));

String wrapped = users.stream()
    .map(User::getEmail)
    .collect(Collectors.joining(", ", "[", "]"));
```

Reducing

```
Optional<Integer> max = numbers.stream()
    .collect(Collectors.reducing(Integer::max));
```

`reducing` is mostly useful as a downstream collector inside `groupingBy`. For top-level reductions, the `Stream.reduce` methods are clearer.

Immutable collection factories (Java 9+)

For constants and small read-only collections.

```
List<String> roles = List.of("admin", "editor", "viewer");
Set<Integer> codes = Set.of(200, 201, 204);
Map<String, Integer> limits = Map.of(
    "free", 100,
    "pro", 10000
);

// More than 10 entries: use ofEntries
Map<String, String> config = Map.ofEntries(
    Map.entry("k1", "v1"),
    Map.entry("k2", "v2"),
    Map.entry("k3", "v3")
);

// Defensive copy of an existing collection
List<String> frozen = List.copyOf(mutableList);
```

Properties: they're truly immutable, so mutation methods throw `UnsupportedOperationException`. They don't allow null elements, keys, or values (throws `NullPointerException`). They use a compact internal representation, smaller than `Collections.unmodifiableList(new ArrayList<>(...))`. `List.copyOf` returns the same instance if the input is already immutable, so no wasted copy.

For new code, `List.of`, `Set.of`, `Map.of` are the modern default for constants.

Synchronized vs concurrent vs immutable

Quick decision table for “which thread-safe collection do I pick”:

Need	Pick
Read-mostly listener list	<code>CopyOnWriteArrayList</code>
General-purpose thread-safe map	<code>ConcurrentHashMap</code>
Sorted thread-safe map	<code>ConcurrentSkipListMap</code>
Producer/consumer queue	<code>ArrayBlockingQueue</code> or <code>LinkedBlockingQueue</code>
Bounded queue with priorities	<code>PriorityBlockingQueue</code>
Lock-free FIFO	<code>ConcurrentLinkedQueue</code>
Truly immutable	<code>List.of</code> , <code>Set.of</code> , <code>Map.of</code>
Read-only view of a mutable collection	<code>Collections.unmodifiable*</code>

Best practices

- Use `ArrayDeque` for queues and stacks in non-concurrent code. Skip `Stack` and `LinkedList` - as-queue.
- Pick `offer` / `poll` / `peek` over `add` / `remove` / `element` unless empty is genuinely an error.
- Reach for `ConcurrentHashMap` first when you need a thread-safe map. Only drop down to `synchronizedMap` for compatibility with old APIs.
- Use `computeIfAbsent` and `merge` on `ConcurrentHashMap` . They’re atomic per key and remove a whole class of races.
- Use `BlockingQueue` for producer/consumer with backpressure. Bound it so a runaway producer can’t OOM you.
- Prefer `List.of` , `Set.of` , `Map.of` for constants over `Collections.unmodifiableList(Arrays.asList(...))` .
- Use `Collectors.toUnmodifiableList()` when collecting a stream meant to be read-only.
- Always pass a merge function to `Collectors.toMap` if duplicates are even remotely possible.
- For “register a listener, then fire events often” patterns, `CopyOnWriteArrayList` is usually right.
- When iterating a synchronized wrapper, wrap the iteration in `synchronized(coll) { ... }` . Forgetting this is a classic bug.

Common gotchas

- `ArrayDeque` doesn't allow nulls. Inserting one throws `NullPointerException`.
- `PriorityQueue` iteration order isn't sorted. Only `poll` gives you sorted output.
- `ConcurrentHashMap` doesn't allow null keys or values. `HashMap` does. Switching from one to the other can surface latent null bugs.
- `Arrays.asList(intArray)` returns `List<int[]>` of size 1, not `List<Integer>`. Use `Arrays.stream(intArray).boxed().toList()` instead.
- `Arrays.asList(...)` returns a fixed-size list. `add` and `remove` throw. Wrap in `new ArrayList<>(...)` if you need mutability.
- `Collectors.toMap` throws on duplicate keys unless you provide a merge function. Easy to miss until production data hits a duplicate.
- `Collectors.toList()` makes no guarantee about mutability. Use `toUnmodifiableList()` if you need read-only, or `toCollection(ArrayList::new)` if you need a specific mutable type.
- Iterating a `synchronizedList` without an external `synchronized` block can throw `ConcurrentModificationException` under concurrent modification.
- `CopyOnWriteArrayList` is great for read-mostly. Writes copy the whole array, so a write-heavy workload will tank performance.
- `ConcurrentLinkedQueue.size()` is $O(n)$ and not atomic with concurrent updates. Don't use it for flow control.
- `SynchronousQueue` has zero capacity. `put` always blocks unless a `take` is already waiting. People sometimes pick it expecting buffering and get deadlocks.
- `DelayQueue` requires elements to implement `Delayed`. Forgetting this gives you a compile error, but the contract on `getDelay` (must be consistent with `compareTo`) is easy to get wrong.
- `Collections.unmodifiableList` returns a view, not a copy. Changes to the underlying list are visible through the wrapper. Use `List.copyOf` for a true immutable snapshot.
- `List.of`, `Set.of`, `Map.of` don't accept nulls. Passing one throws `NullPointerException`, instead of a friendlier error.

Quick reference

Type	Thread-safe	Ordering	Nulls	Notable methods
ArrayDeque	No	Insertion (both ends)	No	addFirst , pollLast , push , pop
LinkedList	No	Insertion	Yes	Queue + Deque + List
PriorityQueue	No	Heap (min)	No	offer , poll , peek
ArrayBlockingQueue	Yes (single lock)	FIFO	No	put , take , bounded
LinkedBlockingQueue	Yes (two locks)	FIFO	No	put , take , optionally bounded
PriorityBlockingQueue	Yes	Heap (min)	No	Unbounded, blocking take
DelayQueue	Yes	Delay expiry	No	Elements implement Delayed
SynchronousQueue	Yes	Direct handoff	No	Zero capacity
LinkedTransferQueue	Yes	FIFO	No	transfer(e) blocks until taken
ConcurrentLinkedQueue	Yes (lock-free)	FIFO	No	Unbounded, non-blocking
ConcurrentLinkedDeque	Yes (lock-free)	Both ends	No	Unbounded, non-blocking
ConcurrentHashMap	Yes (bucket-level)	None	No	compute , merge , weakly consistent iter
ConcurrentSkipListMap	Yes	Sorted (key)	No keys	NavigableMap ops concurrent
ConcurrentSkipListSet	Yes	Sorted	No	Set companion

Type	Thread-safe	Ordering	Nulls	Notable methods
<code>CopyOnWriteArrayList</code>	Yes	Insertion	Yes	Snapshot iterator, expensive writes
<code>CopyOnWriteArraySet</code>	Yes	Insertion	Yes	Backed by COW list
<code>Collections.synchronized*</code>	Yes (whole-coll lock)	Depends on wrapped	Depends	Legacy, manual iter sync needed
<code>List.of</code> / <code>Set.of</code> / <code>Map.of</code>	Yes (immutable)	Insertion / unspecified	No	Compact, throw on mutation